Blocking the First Connection: Where the Time Goes in DPI-Guided XDP Enforcement

Draft for internal review

Shelender Kumar
School of Electrical Engineering and Computer Science (SEECS), NUST
Islamabad, Pakistan

ABSTRACT

Gateways that block domains must turn names seen by deep packet inspection (DPI) into packet drops on IP addresses. We ask how long that takes and whether the first connection to a blocked domain gets through. We built DPI Guard, which couples NFStream/nDPI flow inspection with an eBPF/XDP blocklist, and measured three decision strategies on a Linux testbed. Acting when a flow is exported blocked domains a median of 5.8 s after the ClientHello and never stopped the first connection; on a quiet link, 11 of 30 flows were never acted on. Acting on every packet inside the DPI engine cut the median to 489 ms, a delay we trace to buffered packet capture rather than to classification. A fast path that parses only TLS ClientHellos from an unbuffered, BPF-filtered socket blocked domains 0.81 ms after the ClientHello and stopped 30 of 30 first connections. The XDP hash-map blocklist kept a constant per-packet cost from 1 to 10,000 entries, while linear iptables rules lost 98% of their throughput. We release the system, the testbed and all raw measurements.

KEYWORDS

eBPF, XDP, deep packet inspection, TLS SNI, packet filtering, measurement

1 INTRODUCTION

Network operators block domains, but packets carry IP addresses. A campus, enterprise or home gateway that must stop traffic to `youtube.com` has to learn which server addresses currently serve that name, and then drop their packets fast enough for the policy to hold.

Deep packet inspection solves the first half: libraries such as nDPI [5] read the server name from the TLS ClientHello's Server Name Indication (SNI) [1, 12], from QUIC [8], DNS or the HTTP `Host` header. eBPF and the eXpress Data Path (XDP) [7] solve the second: an XDP program drops packets in the driver, before the kernel allocates socket buffers, and its blocklist can be changed at run time through BPF maps.

This paper studies what happens *between* the two halves: how long it takes from the moment a blocked name appears on the wire until the kernel drops that server's packets, and whether the first connection completes in the meantime. A blocker that lets every first connection through is weak in practice: a page load, an API call or a software update may need only one connection.

We built DPI Guard, an open-source DPI-guided XDP blocker [9], and measured three decision strategies. Our main finding is that the dominant delay is neither DPI nor the kernel, but how packets reach user space:

- Acting when a flow is *exported* waits for the flow's idle timeout, so the first connection always completes (median time-to-block 5.8 s); on a quiet link the flow may never be exported.
- Acting in a *per-packet* DPI hook is faster, but buffered libpcap capture delivers packets in blocks on a timer, adding a uniform 0–1 s delay (median 489 ms).
- A *fast path*, a filtered `AF_PACKET` socket that wakes only for TLS ClientHellos, reduces time-to-block to 0.81 ms; the server's first reply is dropped and every first connection fails.

Contributions. (1) The design and implementation of a DPI-guided XDP domain blocker with a two-path architecture and a header-free user-space controller that drives BPF maps through the `bpf(2)` system call (§3). (2) A measurement of time-to-block for three decision strategies that identifies capture buffering as the dominant delay (§5.1). (3) A comparison of drop cost and blocklist scaling for XDP hash maps, nftables sets and iptables rules (§5.2). (4) A policy model that maps domain rules to IP addresses with exact unblocking and reports collateral blocking (§3).

2 BACKGROUND AND RELATED WORK

Packet capture. User-space monitors read packets through libpcap, which descends from the BSD Packet Filter [10]. On Linux, libpcap uses a memory-mapped `TPACKET_V3` ring that hands *blocks* of packets to user space when a block fills or its timeout expires, trading latency for throughput. Immediate mode [15] removes this delay, but frameworks built on libpcap do not always expose it.

DPI and flow export. nDPI [5] classifies flows and extracts metadata such as the TLS server name. NFStream [2] wraps nDPI in a Python flow-measurement framework that exports a record when a flow ends (idle or active timeout) and runs user plugins on every packet.

Kernel packet filtering. XDP [7] runs verified eBPF programs at the earliest point of the receive path and is used in production for DDoS mitigation [3]. Scholz et al. [14] analysed the performance of eBPF-based filtering, and bpf-iptables [11] showed that rule-by-rule iptables matching degrades with the number of rules while eBPF data structures do not. nftables sets offer hash-based matching inside netfilter, including on the early `netdev` ingress hook.

Name-based blocking and its limits. SNI is sent in clear text in TLS 1.2 and 1.3 [1, 12], which makes it a common signal for filtering and censorship [4]; Encrypted Client Hello (ECH) [13] hides it. Blocking by IP has collateral effects when domains share addresses, and Hoang et al. [6] show how widely domains are co-hosted.

Gap. Prior work evaluates either classification or filtering throughput in isolation. We measure the end-to-end delay between

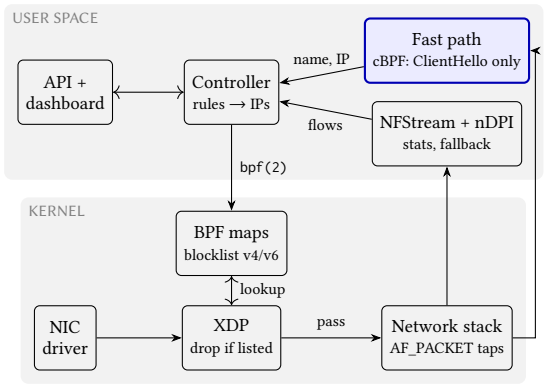


Figure 1: DPI Guard architecture. Two taps observe server names; the controller turns domain rules into IP addresses and writes them to pinned BPF maps that the XDP program consults.

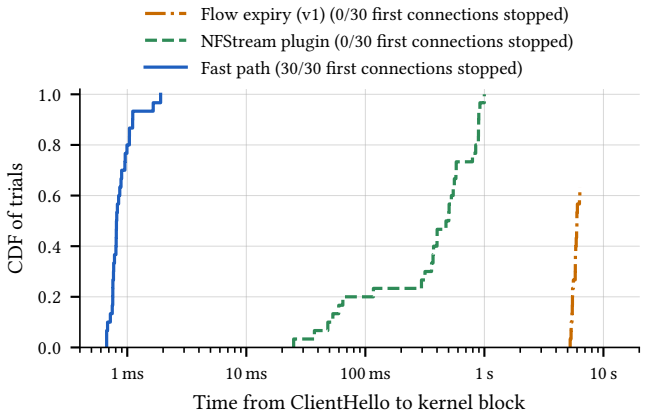


Figure 2: Time from the client's ClientHello to the kernel block, 30 trials per mode. The expiry curve stops at 19/30: the remaining flows were never exported.

classification and enforcement, and whether a DPI-guided blocker stops the *first* connection.

3 DESIGN

DPI Guard splits the work in two: user space decides *which* addresses to block from the names it observes, and an XDP program in the kernel drops their packets (Figure 1).

Enforcement. One XDP program on the monitored interface parses Ethernet, up to two VLAN tags and the IPv4 or IPv6 header, and looks up both the source and destination address in a hash map with 65,536 entries per address family. A hit returns `XDP_DROP` and increments that entry's packet and byte counters; a miss returns `XDP_PASS`. Matching both directions drops a blocked server's replies on an end host and outbound requests on a gateway. Global counters live in a per-CPU array, so the hot path takes no shared locks.

Decision. Two taps feed the controller. The *fast path* is an `AF_PACKET` socket with a 28-instruction classic-BPF filter that accepts only IPv4 TCP segments to port 443 whose payload begins with a TLS handshake record of type ClientHello; user space therefore wakes only for connection openings, parses the SNI and reports (name, server IP) at once. *NFStream with nDPI* inspects all traffic, keeps per-domain statistics, and acts as a fallback for names the fast path cannot see (QUIC, DNS, HTTP, a ClientHello split across segments).

Policy. A rule such as `youtube.com` matches the name and all subdomains; the most specific rule wins. The controller remembers every observed (domain, IP) pair, so adding a rule blocks already-seen addresses at once, and removing one unblocks exactly the addresses no remaining rule needs. When a blocked address also serves an allowed name, the controller reports it as collateral.

Implementation. DPI Guard is about 1,000 lines of Python plus a 128-line XDP program compiled ahead of time with clang and loaded with iproute2. The controller manipulates the pinned maps with five `bpf(2)` commands issued through `ctypes`, so it needs no BCC or kernel headers at run time. A Flask API exposes rules and counters. Twenty-six automated tests, three of which run XDP and nftables on a veth pair inside a network namespace, check enforcement end to end.

4 METHODOLOGY

Testbed. A KVM virtual machine with 4 vCPUs (Intel Xeon, 2.8 GHz), 15 GB RAM, Linux 6.18 and iptables 1.8.10 (nf_tables back end). A sender in a network namespace reaches the host through a veth pair; filters attach to the host-side interface (XDP in native mode). A TLS 1.3 server in the namespace answers one HTTP request per connection.

Time-to-block (§5.1). For each decision mode we run 30 trials. Trial i connects to its own server address with SNI `t`i`.blocked.test`, so earlier blocks never affect later trials. The client records the time just before sending its ClientHello; DPI Guard records when it wrote the IP to the XDP map. Their difference is the *time-to-block*. Every trial is paired with a control connection to an allowed name on another address, which must succeed.

Drop cost and scaling (§5.2). A C sender (`sendmmsg`, batches of 64 UDP packets, 64-byte frames) floods the host for 5 s while pinned to one vCPU. On a veth pair the receive path runs on the sending CPU, so the achieved sender rate measures the whole per-packet path. For drop cost we block the sender's address; for scaling we fill the blocklist with N random addresses that never match, so every packet takes the full no-match path. Each configuration runs 5 times; we report the mean and sample standard deviation.

5 RESULTS

5.1 Time-to-Block

Figure 2 shows the distribution of time-to-block for each mode.

Flow expiry acts only after the connection has ended and stayed idle for the 5 s timeout. Every first connection completed, and the block landed 5.3–6.3 s after the ClientHello. NFStream checks for idle flows only when new packets arrive: the last 11 trials, after which the link went quiet, were never enforced during the 9 s

Table 1: Sender rate with the sender's own address blocked (kpps, 5 runs).

Path	Mean	SD
XDP drop (native)	673	66
nftables `netdev` ingress drop	671	63
iptables `INPUT` drop	582	43
No filter (delivered to socket)	466	35

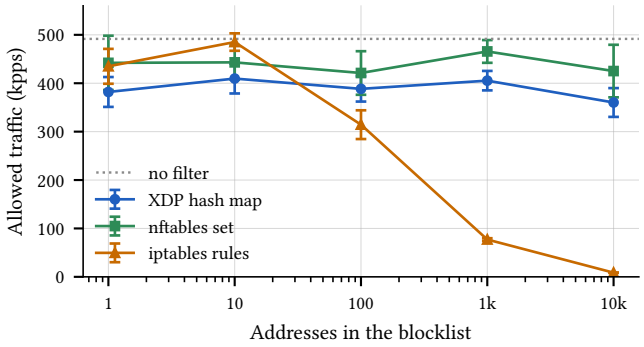


Figure 3: Allowed traffic forwarded while N non-matching addresses are blocked (mean ± SD, 5 runs).

observation window. On a quiet link, export-based enforcement has no upper bound.

The NFStream plugin sees the server name on the connection's fourth packet, yet the block landed 25–999 ms after the ClientHello (median 489 ms). Almost all of this delay lies between a packet's capture timestamp and the plugin call, and it is spread evenly over 0–1 s: the signature of a capture buffer released on a timer. Classification and the map update take well under a millisecond.

The fast path removes the buffer: the block landed 0.67–1.91 ms after the ClientHello (median 0.81 ms; 0.27 ms from frame arrival to map update). The server's first reply was dropped, so all 30 first connections failed. All 90 control connections succeeded.

On our testbed the server answers in well under a millisecond, so even a 25 ms block arrives too late for the plugin. Over the Internet, round-trip times of tens of milliseconds give the fast path a wide margin, while the plugin stops a first connection only when its 0–1 s delay happens to be shorter than the RTT. [TODO: Repeat with emulated server delays of 5–100 ms and plot the fraction of first connections stopped against RTT.]

5.2 Drop Cost and Blocklist Scaling

Drop cost. Table 1: dropping is cheaper than delivering, and on veth XDP and the nftables `netdev` ingress hook perform the same within noise, both about 16% faster than an iptables `INPUT` rule, which runs later in the stack. We expected XDP to lead, but on veth the sender has already allocated the socket buffer that XDP normally avoids, and attaching XDP adds a NAPI hand-off. This result shows that the XDP enforcer costs no more than the best netfilter alternative here; its advantage on physical NICs [7, 14] remains to be measured on our hardware.

Scaling. Figure 3: with hash-based lookups the blocklist size does not matter; XDP's map and the nftables set forward allowed traffic at the same rate with 1 or 10,000 entries. iptables evaluates its rules one by one, so every allowed packet pays for every rule: at 10,000 addresses it forwards 8.6 kpps, 2% of its single-rule rate, consistent with the linear degradation reported for iptables [11]. Attaching XDP to a veth interface costs about 20% of forwarding rate even when nothing is dropped, because veth then processes frames through an extra NAPI step; this overhead is specific to veth.

6 DISCUSSION

Where the time goes. In a DPI-guided blocker, the kernel and the classifier are fast; the delay comes from how packets reach the classifier. Export-on-expiry ties enforcement to the flow's end and to later traffic. A per-packet hook fixes the first problem but inherits the capture library's buffering. A narrow, unbuffered tap for the one message that names the server fixes both. Latency-critical decisions in monitoring systems need their own capture path.

An operational lesson. NFStream's meter processes ignore `SIGTERM`. When our controller exited, they survived as orphans that kept capturing. While they ran, packets from blocked addresses intermittently bypassed the native-mode XDP program on our veth testbed and 0/30 first connections were stopped; after the controller force-killed its children, 30/30 were stopped. We have not determined the kernel mechanism; capture processes must be supervised as strictly as the filter itself.

Limitations. *Collateral blocking:* enforcement is per address, so an allowed name sharing a CDN address with a blocked one is blocked too; DPI Guard reports but cannot avoid this [6]. *Hidden names:* ECH [13] hides the SNI and encrypted DNS hides the query. *Large ClientHellos:* post-quantum hybrid key shares can push the ClientHello past one TCP segment; the fast path parses only the first segment and falls back to NFStream otherwise. *QUIC:* the fast path handles TCP only; QUIC Initial packets are encrypted with keys derived from public values, so a QUIC fast path is feasible but not yet built.

Threats to validity. All measurements come from one virtual machine on a veth pair with sub-millisecond RTT, which hides XDP's main advantage and adds a NAPI step of its own. IPv6 was disabled in the kernel, so only IPv4 was measured. Client and controller timestamps share one clock. Samples are 30 trials per time-to-block mode and 5 runs per throughput point.

7 CONCLUSION

A DPI-guided blocker is only as fast as the path that carries packets to its classifier. Acting on exported flows took 5.8 s and never stopped a first connection; acting inside the DPI engine took 489 ms because of capture buffering; a ClientHello fast path took 0.81 ms and stopped all first connections, while the XDP blocklist kept a constant cost from 1 to 10,000 entries. Future work: bare-metal runs with native-XDP NICs, collateral measurements on real CDN-hosted domains, a QUIC fast path, and connection-level enforcement to avoid collateral blocking. Code, testbed and data are available [9].

REFERENCES

[1] Donald Eastlake 3rd. 2011. Transport Layer Security (TLS) Extensions: Extension Definitions. RFC 6066.

[2] Zied Aouini and Adrián Pekár. 2022. NFStream: A Flexible Network Data Analysis Framework. *Computer Networks* 204 (2022), 108719.

[3] Gilberto Bertin. 2017. XDP in Practice: Integrating XDP into our DDoS Mitigation Pipeline. In *Proceedings of Netdev 2.1, The Technical Conference on Linux Networking*.

[4] Zimo Chai, Amirhossein Ghafari, and Amir Houmansadr. 2019. On the Importance of Encrypted-SNI (ESNI) to Censorship Circumvention. In *9th USENIX Workshop on Free and Open Communications on the Internet (FOCI)*. USENIX.

[5] Luca Deri, Maurizio Martinelli, Tomasz Bujlow, and Alfredo Cardigliano. 2014. nDPI: Open-Source High-Speed Deep Packet Inspection. In *International Wireless Communications and Mobile Computing Conference (IWCMC)*. IEEE, 617–622. https://doi.org/10.1109/IWCMC.2014.6906427

[6] Nguyen Phong Hoang, Arian Akhavan Niaki, Nikita Borisov, Phillipa Gill, and Michalis Polychronakis. 2020. Assessing the Privacy Benefits of Domain Name Encryption. In *Proceedings of the 15th ACM Asia Conference on Computer and Communications Security (ASIA CCS)*. ACM.

[7] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking EXperiments and Technologies (CoNEXT)*. ACM, 54–66.

[8] Jana Iyengar and Martin Thomson. 2021. QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000.

[9] Shelender Kumar. 2026. DPI Guard: source code, testbed scripts and measurement data. https://github.com/shelenderrkumar/Network-Monitoring-App.

[10] Steven McCanne and Van Jacobson. 1993. The BSD Packet Filter: A New Architecture for User-Level Packet Capture. In *USENIX Winter 1993 Conference*. USENIX.

[11] Sebastiano Miano, Matteo Bertrone, Fulvio Risso, Mauricio Vásquez Bernal, Yunsong Lu, and Jianwen Pi. 2019. Securing Linux with a Faster and Scalable iptables. *ACM SIGCOMM Computer Communication Review* 49, 3 (2019), 3–17. https://doi.org/10.1145/3371927.3371929

[12] Eric Rescorla. 2018. The Transport Layer Security (TLS) Protocol Version 1.3. RFC 8446.

[13] Eric Rescorla, Kazuho Oku, Nick Sullivan, and Christopher A. Wood. 2025. TLS Encrypted Client Hello. IETF Internet-Draft draft-ietf-tls-esni-25. Work in progress; replace with the RFC number once published.

[14] Dominik Scholz, Daniel Raumer, Paul Emmerich, Alexander Kurtz, Krzysztof Lesiak, and Georg Carle. 2018. Performance Implications of Packet Filtering with Linux eBPF. In *30th International Teletraffic Congress (ITC 30)*. IEEE. https://doi.org/10.1109/ITC30.2018.00039

[15] The Tcpdump Group. [n. d.]. pcap_set_immediate_mode(3PCAP) manual page. https://www.tcpdump.org/manpages/pcap_set_immediate_mode.3pcap.html. Accessed October 2026.